

MERN Stack Development SOP

1. Project Structure

Backend Structure

```
backend/
├── controllers/
├── models/
│   └── zindex.js
├── routes/
├── validations/
├── middleware/
├── utils/
├── uploads/
├── config/
├── public/
└── app.js
```

Frontend Structure

```
frontend/
├── src/
│   ├── pages/
│   ├── components/
│   ├── services/
│   ├── utils/
│   └── assets/
```

2. File Naming Standards

1. Route files must follow module-wise naming.
2. Controller files must follow module-wise naming.
3. Model files must follow module-wise naming.
4. All models must be exported through `models/zindex.js`.
5. Use consistent camelCase naming conventions.

3. Backend Standards

1. Follow proper separation of Routes, Controllers, Models, Validations, Middleware, and Utils.
 2. All APIs must have validation before controller execution.
 3. Use centralized response handling.
 4. Use proper error handling in all APIs.
 5. Avoid duplicate code and create reusable utility functions whenever possible.
 6. Maintain proper comments only where required.
-

4. Frontend Standards

1. All API calls must be managed from the Services folder.
 2. Components and Pages must not directly call APIs.
 3. Create reusable components whenever possible.
 4. Maintain proper route management.
 5. Keep business logic separate from UI components.
 6. Use common utility functions for repeated operations.
-

5. Environment Configuration

1. Maintain both `.env` and `.env.development`.
 2. Development environment must use `.env.development`.
 3. Production build must use `.env`.
 4. Production environment must always contain live URLs, domains, and ports.
 5. No hardcoded URLs are allowed in the codebase.
-

6. Frontend Build & Deployment

1. Frontend build must be generated using production configuration.
 2. Frontend build must be deployed inside `backend/public`.
 3. Backend should serve the frontend build.
 4. Refreshing any frontend route should not return a 404 error.
-

7. UI/UX Standards

1. Maintain consistent colors throughout the application.
2. Maintain consistent typography and font sizes.
3. Maintain consistent button styles.

4. Maintain consistent form designs.
 5. Maintain consistent table and list designs.
 6. Maintain consistent spacing, margins, and paddings.
 7. Maintain consistent popup, drawer, and modal designs.
 8. Follow the approved design system across all pages.
-

8. Popup Standards

1. Popup header must remain fixed.
 2. Popup footer must remain fixed.
 3. Only popup content should be scrollable.
 4. Mobile popups should be Bottom Sheet or Full Screen Popup.
 5. Popup UI should remain consistent across the project.
-

9. Form Validation Standards

1. All forms must have frontend validation before API submission.
 2. Validation messages must be displayed directly below the respective field.
 3. Invalid fields must show a red border.
 4. Invalid fields must automatically receive focus when submitting the form.
 5. Validation messages must be user-friendly and clearly explain the issue.
 6. Forms should prevent API calls until validation passes.
 7. Backend validation is mandatory even if frontend validation exists.
-

10. API Response & Error Message Standards

1. Every API must return proper success and error messages from the backend.
2. Frontend must display the exact message received from the backend.
3. Do not use hardcoded error messages such as:
 - Something Went Wrong
 - Server Error
 - Unknown Error
4. Generic messages should only be shown when:
 - API is not reachable.
 - Network connection fails.
 - Backend is not responding.
 - Unexpected system failure occurs.
5. All validation messages must come from backend responses whenever applicable.
6. Success messages must also come from backend responses.
7. API response formats must remain consistent throughout the project.

11. Image Upload Standards

1. Use Multer for all file uploads.
 2. Store uploads inside the uploads folder.
 3. Maintain module-wise upload folders.
 4. Store only the upload path in the database.
 5. Frontend must append the backend domain before displaying images.
 6. Validate file type and size before upload.
-

12. Authentication & Route Protection

1. Private routes must require authentication.
 2. Unauthenticated users must be redirected to Login.
 3. Authenticated users should not access Login or Register pages.
 4. Invalid routes must redirect to 404 Page.
 5. Proper session and token validation must be implemented.
-

13. Basic Mobile Responsiveness

1. it should not overflow in the mobile
 2. Every page must be mobile responsive.
 3. Every form must be mobile responsive.
 4. Every table must be mobile responsive.
 5. Every popup must be mobile responsive.
 6. All pages must be tested on Mobile, Tablet, and Desktop screens.
-

14. Testing Flow

Backend Testing

1. Verify request validations.
2. Verify API success responses.
3. Verify API error responses.
4. Verify authentication and authorization.
5. Verify database operations.
6. Verify image uploads.
7. Verify pagination, filters, and search functionality.
8. Verify proper backend messages for all scenarios.

Frontend Testing

1. Verify all pages are loading correctly.
2. Verify all forms are working.
3. Verify all validation messages.
4. Verify field focus and red border behavior.
5. Verify all API integrations.
6. Verify route redirections.
7. Verify role-based access.
8. Verify image rendering.
9. Verify responsive layouts.
10. Verify popup behavior.
11. Verify backend messages are displayed properly.

Deployment Testing

1. Verify production build generation.
 2. Verify frontend serving from backend public folder.
 3. Verify environment variables.
 4. Verify domain configuration.
 5. Verify SSL functionality.
 6. Verify all routes after deployment.
 7. Verify image uploads in production.
 8. Verify API URLs are pointing to production servers.
-

15. Final Delivery Checklist

1. Folder structure followed.
2. Naming conventions followed.
3. API validations implemented.
4. Frontend validations implemented.
5. Services-based API management implemented.
6. Environment configuration completed.
7. Authentication implemented.
8. Route protection implemented.
9. Image upload implemented.
10. Mobile responsiveness completed.
11. Testing completed.
12. Production build verified.
13. Proper backend messages implemented.
14. No critical bugs pending.
15. Code reviewed and cleaned before delivery.